INITIATION PYTHON · JOUR 4 SUR 5 · JEUDI 5 NOVEMBRE

Jeudi · Dictionnaires, modules, fichiers, erreurs

Jusqu'ici, vos programmes oubliaient tout en s'arrêtant, et plantaient à la première saisie bizarre. Aujourd'hui, on range les données par nom (les dictionnaires), on utilise le code des autres (les modules), on lit et on écrit des fichiers, et on rattrape les erreurs au lieu de les subir.

1. Les dictionnaires

Une liste range des valeurs par **position** : `notes[0]`, `notes[1]`. Un dictionnaire les range par **clé** : un nom, un mot, n'importe quel texte.

Dans l'interpréteur Python

```
>>> ages = {"Alice": 32, "Bob": 25}
>>> ages["Alice"]
32
>>> ages["Chloé"] = 28
>>> ages
{'Alice': 32, 'Bob': 25, 'Chloé': 28}
>>> "Bob" in ages
True
>>> len(ages)
3
>>> ages["Zoé"]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'Zoé'
>>> ages.get("Zoé", 0)
0
>>> ages["Bob"] = 26
>>> del ages["Alice"]
>>> ages
{'Bob': 26, 'Chloé': 28}
```

- Un dictionnaire s'écrit entre accolades, avec des paires `clé: valeur`.
- `d[cle]` lit la valeur. Si la clé n'existe pas : `KeyError`.
- `d[cle] = valeur` ajoute la paire, ou remplace la valeur si la clé existe déjà. Une clé n'apparaît qu'une fois.
- `cle in d` teste si la clé existe. `d.get(cle, defaut)` lit la valeur, ou renvoie `defaut` si la clé n'existe pas, sans erreur.

Parcourir un dictionnaire avec `.items()`, qui donne la clé et la valeur à chaque tour :

repertoire.py

```python
telephones = {"Alice": "06 12 34 56 78", "Bob": "07 98 76 54 32", "Chloé": "06 55 44 33 22"}

for nom, numero in telephones.items():
    print(f"{nom:8} {numero}")
```

Dans le terminal

```
$ python3 repertoire.py
Alice    06 12 34 56 78
Bob      07 98 76 54 32
Chloé    06 55 44 33 22
```

Le `:8` dans la f-string réserve 8 caractères pour le nom : les numéros sont alignés.

Le patron du compteur. C'est l'usage le plus courant d'un dictionnaire : compter combien de fois chaque chose apparaît.

compter.py

```
phrase = "le chat et le chien et le lapin"
compte = {}
for mot in phrase.split():
    compte[mot] = compte.get(mot, 0) + 1
print(compte)
```

Dans le terminal

```
$ python3 compter.py
{'le': 3, 'chat': 1, 'et': 2, 'chien': 1, 'lapin': 1}
```

À chaque mot, on lit son compte actuel (0 s'il n'a jamais été vu), on ajoute 1, et on range le résultat.

> **En bash** : `declare -A ages`, `ages[Alice]=32`, `${ages[Alice]}`, `for nom in "${!ages[@]}"`. En Python : `ages = {}`, `ages["Alice"] = 32`, `ages["Alice"]`, `for nom, age in ages.items():`.

2. Renvoyer plusieurs valeurs : les tuples

Un **tuple** ressemble à une liste, entre parenthèses, mais ne se modifie pas. Son usage le plus courant : faire renvoyer plusieurs valeurs à une fonction.

min_max.py

```
def min_max(liste):
    plus_petit = liste[0]
    plus_grand = liste[0]
    for x in liste:
        if x < plus_petit:
            plus_petit = x
        if x > plus_grand:
            plus_grand = x
    return plus_petit, plus_grand


bas, haut = min_max([12, 8, 15, 10, 17])
print(f"De {bas} à {haut}")
```

Dans le terminal

```
$ python3 min_max.py
De 8 à 17
```

`return plus_petit, plus_grand` renvoie un tuple de deux valeurs, et `bas, haut = …` les range dans deux variables d'un coup.

3. Les modules

Python est livré avec des centaines de **modules** : du code déjà écrit, qu'on importe au début du fichier.

modules.py

```python
import math
import random

print(math.sqrt(144))
print(round(math.pi, 4))
print(random.randint(1, 6))
print(random.choice(["pierre", "feuille", "ciseaux"]))

cartes = ["as", "roi", "dame", "valet"]
random.shuffle(cartes)
print(cartes)
```

Un exemple de résultat : le hasard change à chaque lancement

```
$ python3 modules.py
12.0
3.1416
4
ciseaux
['valet', 'as', 'roi', 'dame']
```

Module	Fonction	Ce qu'elle fait
`random`	`random.randint(a, b)`	un entier au hasard entre `a` et `b`, **inclus**
	`random.choice(liste)`	un élément au hasard
	`random.shuffle(liste)`	mélange la liste sur place
`math`	`math.sqrt(x)`, `math.pi`	racine carrée, π
`time`	`time.sleep(2)`	attend 2 secondes
`sys`	`sys.argv`	les arguments du programme (section 6)

On peut aussi importer une seule fonction : `from math import sqrt`, puis `sqrt(144)` sans le `math.` devant. Pour lire l'aide d'un module dans l'interpréteur : `help(random.randint)`.

4. Lire et écrire des fichiers

Prenons un fichier de notes, une ligne par élève, les champs séparés par des points-virgules :

notes.txt

```
Alice;14
Bob;9
Chloé;17
David;11
```

lire_notes.py

```python
with open("notes.txt", encoding="utf-8") as fichier:
    for ligne in fichier:
        prenom, note = ligne.strip().split(";")
        note = int(note)
        if note >= 10:
            print(f"{prenom} : {note}, admis")
        else:
            print(f"{prenom} : {note}, recalé")
```

Dans le terminal

```
$ python3 lire_notes.py
Alice : 14, admis
Bob : 9, recalé
Chloé : 17, admis
David : 11, admis
```

- `open(nom, encoding="utf-8")` ouvre le fichier en lecture. Le `with … as fichier:` le referme tout seul à la fin du bloc, même en cas d'erreur.
- `for ligne in fichier:` lit le fichier ligne par ligne.
- Chaque ligne se termine par un retour à la ligne invisible, `\n` : `.strip()` le retire.
- `.split(";")` découpe la ligne aux points-virgules, et `prenom, note = …` range les deux morceaux.
- Tout ce qu'on lit dans un fichier est du **texte** : d'où le `int(note)`.

Écrire : on ouvre avec `"w"` (*write*, le fichier est vidé puis réécrit) ou `"a"` (*append*, on ajoute à la fin).

ecrire.py

```python
with open("journal.txt", "w", encoding="utf-8") as f:
    f.write("Première ligne\n")
    f.write("Deuxième ligne\n")

with open("journal.txt", "a", encoding="utf-8") as f:
    f.write("Ajoutée à la fin\n")

with open("journal.txt", encoding="utf-8") as f:
    print(f.read())
```

Dans le terminal

```
$ python3 ecrire.py
Première ligne
Deuxième ligne
Ajoutée à la fin
```

`write` n'ajoute pas de retour à la ligne : il faut mettre le `\n` soi-même. `f.read()` lit tout le fichier d'un coup, dans une seule chaîne.

> **En bash** : `while IFS=';' read -r prenom note; do … done < notes.txt` pour lire, `echo "…" > f` et `>> f` pour écrire. En Python : `open` avec `"w"` pour `>`, `"a"` pour `>>`.

5. Rattraper les erreurs : `try` et `except`

Depuis lundi, une saisie de travers fait planter le programme avec une `ValueError`. On peut l'intercepter :

essai_try.py

```python
saisie = input("Votre âge : ")
try:
    age = int(saisie)
    print(f"Dans 10 ans : {age + 10} ans")
except ValueError:
    print(f"« {saisie} » n'est pas un nombre entier.")
```

Deux essais

```
$ python3 essai_try.py
Votre âge : douze
« douze » n'est pas un nombre entier.
$ python3 essai_try.py
Votre âge : 12
Dans 10 ans : 22 ans
```

Python essaie le bloc `try`. Si une `ValueError` se produit, il saute directement au bloc `except`, au lieu de planter. Sinon, le bloc `except` est ignoré.

Combiné avec une boucle, on obtient une saisie qui insiste jusqu'à avoir un nombre valable. C'est une fonction à garder sous la main toute votre vie de développeur :

demander_entier.py

```python
def demander_entier(question):
    """Redemande jusqu'à obtenir un nombre entier, et le renvoie."""
    while True:
        saisie = input(question)
        try:
            return int(saisie)
        except ValueError:
            print("Il faut un nombre entier, recommencez.")


n = demander_entier("Un nombre : ")
print(f"Le double de {n} est {2 * n}.")
```

On tape abc, 2.5, rien, puis 42

```
$ python3 demander_entier.py
Un nombre : abc
Il faut un nombre entier, recommencez.
Un nombre : 2.5
Il faut un nombre entier, recommencez.
Un nombre :
Il faut un nombre entier, recommencez.
Un nombre : 42
Le double de 42 est 84.
```

Le `return` à l'intérieur du `try` sort de la fonction, et donc de la boucle, dès que la conversion réussit.

Même chose pour un fichier qui n'existe pas :

fichier_absent.py

```python
try:
    with open("inexistant.txt", encoding="utf-8") as f:
        print(f.read())
except FileNotFoundError:
    print("Le fichier inexistant.txt est introuvable.")
```

Dans le terminal

```
$ python3 fichier_absent.py
Le fichier inexistant.txt est introuvable.
```

Erreur	Quand
`ValueError`	`int("abc")` : la valeur n'a pas le bon format
`TypeError`	`"42" + 1` : on mélange des types incompatibles
`IndexError`	`liste[10]` dans une liste de 4 éléments
`KeyError`	`d["Zoé"]` : la clé n'existe pas
`ZeroDivisionError`	`10 / 0`
`FileNotFoundError`	`open("inexistant.txt")`
`NameError`	une variable mal orthographiée, ou pas encore créée

> **Piège.** `except:` tout seul, sans nom d'erreur, attrape **toutes** les erreurs, y compris vos fautes de frappe et le `Ctrl+C`. Le programme a l'air de marcher, mais cache ses bugs. Nommez toujours l'erreur que vous attendez.

6. Les arguments du programme : `sys.argv`

salut.py

```python
import sys

print(sys.argv)
if len(sys.argv) < 2:
    print("Usage : python3 salut.py prénom")
    sys.exit(1)
print(f"Salut {sys.argv[1]} !")
```

Trois lancements

```
$ python3 salut.py Alice
['salut.py', 'Alice']
Salut Alice !
$ python3 salut.py Alice Bob
['salut.py', 'Alice', 'Bob']
Salut Alice !
$ python3 salut.py
['salut.py']
Usage : python3 salut.py prénom
```

`sys.argv` est une liste : le nom du programme en case 0, puis les arguments. `sys.argv[1]` correspond au `$1` de bash, et `sys.exit(1)` au `exit 1`.

Exercices du jeudi

Les fichiers de données sont dans ce support : recopiez-les dans votre dossier, avec VS Code.

J1. Compter les lettres NIVEAU 1

Demandez un mot et affichez combien de fois chaque lettre apparaît, avec un dictionnaire et le patron du compteur. Pour `"abracadabra"` : a 5 fois, b 2 fois, r 2 fois, c 1 fois, d 1 fois.

J2. Le répertoire NIVEAU 1

Un programme qui affiche un menu en boucle : `1. Ajouter`, `2. Chercher`, `3. Lister`, `4. Quitter`. Les contacts sont rangés dans un dictionnaire `nom → téléphone`. Chercher un nom absent doit afficher « Inconnu », pas planter.

J3. Le petit traducteur NIVEAU 2

Avec le dictionnaire suivant, traduisez mot à mot une phrase tapée par l'utilisateur. Un mot inconnu s'affiche entre crochets.

```
dico = {"le": "the", "chat": "cat", "chien": "dog", "mange": "eats", "la": "the", "souris": "mouse",
"petit": "small"}
```

`le petit chat mange la souris verte` donne `the small cat eats the mouse [verte]`. Remarquez que la traduction mot à mot a ses limites : en anglais, l'adjectif va avant le nom.

J4. Le juste prix NIVEAU 2

L'ordinateur choisit un nombre entre 1 et 100 avec `random.randint`. Le joueur propose des nombres, l'ordinateur répond « C'est plus » ou « C'est moins », jusqu'à ce qu'il trouve. Affichez le nombre de coups. Utilisez `demander_entier` pour que le jeu ne plante jamais.

Quelle stratégie trouve à coup sûr en 7 coups au maximum ? (On en reparle demain.)

J5. Le carnet de notes NIVEAU 2

Le fichier `eleves.txt` contient une ligne par élève, avec un nombre de notes variable :

eleves.txt

```
Alice;14;16;12
Bob;9;11
Chloé;17;15;18;16
David;8;12;10
```

Lisez-le et affichez la moyenne de chaque élève, avec une décimale, puis le nom de l'élève qui a la meilleure moyenne. Astuce : après `champs = ligne.strip().split(";")`, `champs[0]` est le prénom et `champs[1:]` la liste des notes, encore en texte.

J6. Les mots les plus fréquents NIVEAU 3

Comptez les mots du fichier `texte.txt`, sans tenir compte des majuscules ni de la ponctuation, puis affichez les 5 plus fréquents avec leur nombre d'apparitions.

texte.txt

```
Le chat dort sur le canapé. Le chien regarde le chat.
Le chat se réveille, le chien aboie, et le chat s'enfuit.
Moralité : le chien et le chat ne sont pas amis.
```

Pour retirer la ponctuation : remplacez chaque signe par un espace avec `.replace()`. Pour trier les mots du plus fréquent au moins fréquent : `sorted(compte, key=compte.get, reverse=True)`.

J7. Le quiz NIVEAU 3

Le fichier `questions.txt` contient une question et sa réponse par ligne :

questions.txt

```
Combien font 7 x 8 ?;56
Quelle commande Git envoie les commits sur GitHub ?;push
Quel mot-clé définit une fonction en Python ?;def
Combien vaut 17 // 5 ?;3
```

Posez les questions dans un ordre aléatoire, comptez les bonnes réponses (sans tenir compte des majuscules ni des espaces autour), affichez le score. Puis demandez le prénom du joueur et **ajoutez** une ligne `prénom;score` à la fin d'un fichier `scores.txt`.

J8. `wc` en Python NIVEAU 3

Réécrivez la commande `wc` : `python3 wc.py texte.txt` affiche le nombre de lignes, de mots et de caractères du fichier. Sans argument, affichez l'usage. Avec un fichier qui n'existe pas, un message clair, pas un traceback. Comparez vos résultats avec la vraie commande : `wc texte.txt`.

Mémo du jeudi

Je veux…	En bash	En Python
créer un dictionnaire	`declare -A d`	`d = {}`
ranger, lire	`d[cle]=v`, `${d[cle]}`	`d[cle] = v`, `d[cle]`
lire sans risque		`d.get(cle, defaut)`
parcourir	`for k in "${!d[@]}"`	`for k, v in d.items():`
compter		`d[x] = d.get(x, 0) + 1`
importer un module	`source fichier.sh`	`import random`
hasard	`$(( RANDOM % 6 + 1 ))`	`random.randint(1, 6)`
lire un fichier	`while read -r l; do … done < f`	`with open(f, encoding="utf-8") as fic:` + `for l in fic:`
écrire, ajouter	`>`, `>>`	`open(f, "w")`, `open(f, "a")`
découper une ligne	`IFS=';' read -r a b`	`a, b = ligne.strip().split(";")`
rattraper une erreur	`if ! commande; then …`	`try:` … `except ValueError:`
arguments	`$1`, `$#`	`sys.argv[1]`, `len(sys.argv) - 1`
quitter avec un code	`exit 1`	`sys.exit(1)`